AE646: Final Report (Stage 3)

Optimising a Fourier Neural Operator for Parametric Darcy Flow: Recipe, Architecture, Data, Resolution, Latency and Comparison with a Finite-Volume Solver

Team PINNacles
Aryamann Srivastava, Varun Sathaye, Atishay Jain, Vedant S. Tiwari
Department of Aerospace Engineering, IIT Kanpur

November 2026

Abstract

We study a Fourier Neural Operator (FNO) and an MLP baseline as surrogates for the 2D Darcy flow equation $-\nabla\cdot(\kappa\nabla u) = 1$ on real PDEBench data (900/100/200 train/validation/test split, 64×64). Stage 2 reported FNO 0.052 vs. MLP 0.082 mean relative L_2 error. In Stage 3 we show that most of the remaining error was *optimisation*, not capacity: replacing the Stage 2 recipe (MSE loss, a learning rate that restarted every 200 steps, no augmentation) by a relative-L_2 loss, a single cosine decay and dihedral-symmetry (D4) augmentation halves the FNO error at an identical 5 700-step budget ($0.052 \to 0.025$; 3 seeds) and helps the MLP too ($0.080 \to 0.045$). Validation-only selection over modes, width, depth, steps and optimiser settings gives a final FNO (9.5 M parameters) with test error 0.0202 ± 0.0004 (5 seeds) against 0.0415 ± 0.0009 for an equally tuned 100 M-parameter MLP: the FNO advantage *grows* from 1.6× to 2.05×. The FNO error plateaus at ≈ 0.02 for $\geq$ 5,000 training samples and is nearly insensitive to the number of Fourier modes (≥ 4). Against a properly implemented vectorised finite-volume solver, calibrated on validation data, the FNO is more accurate than the solver at the same resolution (0.020 vs. 0.044 at 64×64) and 5–35× faster on a GPU, but *not* faster on a single CPU core. We correct the Stage 2 speed-up claim (1443× vs. 1030 ms), which used an inefficient reference solver. Physical diagnostics (PDE residual, boundary values, positivity, error vs. interface distance and permeability composition) show that the residual error is concentrated in a few low-contrast/uniform-κ fields with large pressure amplitude.

1 Introduction and Scope

The physical problem is steady single-phase flow in a heterogeneous porous medium: $-\nabla\cdot(\kappa\nabla u) = f$ on $(0,1)^2$ with $u = 0$ on the boundary and $f = 1$, where $\kappa \in \{0.1, 1.0\}$ is a piecewise-constant permeability and u the pressure. The SciML task is to learn the operator $\mathcal{G} : \kappa \mapsto u$. Baseline: an MLP; SciML model: FNO [1]; data: PDEBench 2D Darcy ($\beta = 1$) [2].

Stage 2 delivered the pipeline, EDA, preprocessing/baseline ablations and a first MLP-vs-FNO comparison. Stage 3 asks: *how good can the FNO be made, what limits it, what does it cost, and how does it compare with a classical solver?* We answer with five controlled studies (§4–§7) and a physical analysis (§8). Two further studies, added after the final presentation on the instructor's suggestions, test robustness to noisy training labels (§9) and a self-adjusting sample weighting aimed at the hardest fields (§10). Every number below comes from a run whose configuration and metrics are stored in `results/stage3/`.

2 Data, Protocol and Metric

The split is unchanged from Stage 2 (seed 42): 900 train, 100 validation, 200 test samples out of 1 200 drawn from the 10 000 real PDEBench fields; inputs are κ plus (x, y) coordinate channels, standardised with training statistics; the 128×128 fields are stride-2 subsampled to 64×64 (§6 also uses 32 and 128). The other 8 800 real samples were never used in Stage 1–2 and appear only in the data-scaling study. **All hyper-parameter and design choices are made on the validation set; the test set is evaluated once per run.** The metric is the per-sample relative L_2 error in physical (de-normalised) units,

$$\mathrm{err}_i = \frac{\|\hat{u}_i - u_i\|_2}{\|u_i\|_2} = \frac{\sqrt{\sum_j (\hat{u}_{ij} - u_{ij})^2}}{\sqrt{\sum_j u_{ij}^2}},$$

where i indexes the map and j the pixels (numerator and denominator are summed separately over the pixels of one map), averaged over the test samples. The relative-L_2 training loss is the same quantity averaged over a batch of $B = 16$ maps, $\mathcal{L} = \frac{1}{B} \sum_{i=1}^{B} \mathrm{err}_i$. Unless stated otherwise, "±" is the standard deviation over seeds.

Symmetry. The data are statistically invariant under the dihedral group D4 (rotations/reflections of the square), and the PDE with $f = 1$ and homogeneous Dirichlet conditions is exactly equivariant. We therefore (i) augment training with a random D4 element per batch, applied at 128×128 *before* sub-sampling (rotating an already sub-sampled field would shift the grid phase and is not valid), and (ii) optionally average predictions over the group at test time (TTA; at 64×64 only {identity, transpose} are phase-consistent, so TTA there is a 2-element average). The Stage 2 protocol (100 epochs = 5 700 steps, batch 16) is kept as the reference *step budget*.

3 Models

FNO2d: lifting $3 \to w$; L Fourier layers, each combining a spectral convolution (rfft2, learned complex weights on the lowest $m \times m$ modes of two frequency bands, irfft2), a 1×1 convolution bypass, layer normalisation and GELU; projection $w \to 128 \to 1$. Stage 2 configuration: $m = 12$, $w = 64$, $L = 4$ (4.74 M parameters). **MLP**: flatten $64^2 \cdot 3$, three hidden layers, GELU, output 64^2 (Stage 2: 3×2048, 42 M parameters). Both use AdamW.

4 Study 1: What Limited Stage 2? The Training Recipe

While designing Stage 3 we found two properties of the Stage 2 recipe: the loss was MSE on standardised pressure (whereas the reported metric is a *relative* error, so low-amplitude samples were under-weighted), and the cosine schedule was stepped per batch with $T_{\max}$ equal to the number of epochs, so the learning rate cycled $10^{-3} \to 0$ every 200 steps (≈28 cycles) instead of decaying once. We reproduced this recipe exactly (row 1 of Table 1: 0.0524 ± 0.0018, matching the reported 0.052) and ablated each change at the same 5 700-step budget, three seeds each.

Table 1: Recipe ablation, 5 700 steps, test relative L_2 (mean ± sd, 3 seeds). TTA = 2-element symmetrisation.

schedule	loss	aug.	FNO (4.7 M)	FNO + TTA	MLP (42 M)
restarting (Stage 2)	MSE	none	0.0524 ± 0.0018	0.0482	0.0802 ± 0.0031
single cosine	MSE	none	0.0485 ± 0.0008	0.0444	0.0719 ± 0.0010
restarting	rel-L_2	none	0.0389 ± 0.0002	0.0366	0.0630 ± 0.0017
restarting	MSE	D4	0.0451 ± 0.0028	0.0416	0.0817 ± 0.0008
single cosine	rel-L_2	none	0.0384 ± 0.0004	0.0359	0.0612 ± 0.0008
restarting	rel-L_2	D4	0.0286 ± 0.0001	0.0266	0.0518 ± 0.0010
single cosine	MSE	D4	0.0369 ± 0.0004	0.0343	0.0649 ± 0.0004
single cosine	**rel-L_2**	**D4**	$\mathbf{0.0249 \pm 0.0002}$	**0.0237**	$\mathbf{0.0454 \pm 0.0003}$

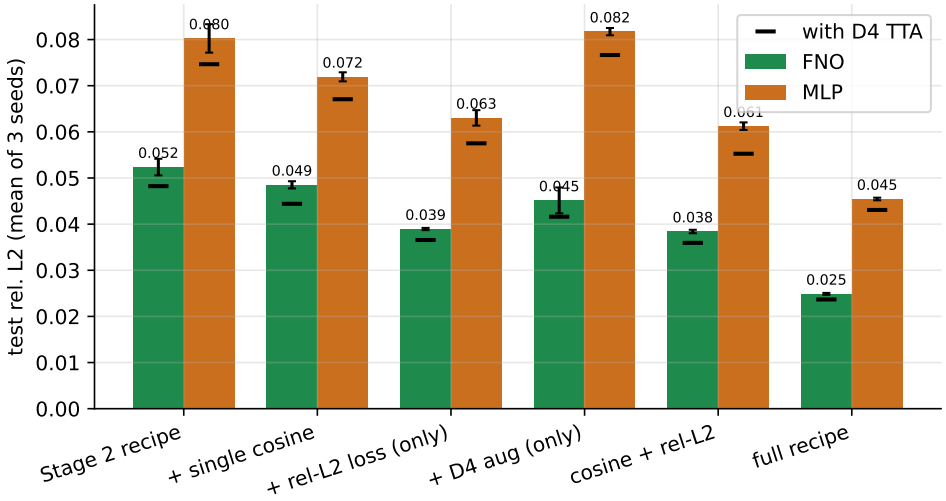


Figure 1: Recipe ablation (bars: test error, error bars: sd over 3 seeds; black ticks: with TTA).

The relative-L_2 loss is the largest single gain (-26% for the FNO), D4 augmentation adds a further -35% on top of it, and the schedule fix contributes $\approx -7\%$; the full recipe halves the FNO error and reduces the MLP error by 43% at identical cost. D4 augmentation *alone* does not help under the MSE loss and the restarting schedule (MLP: 0.0817 vs. 0.0802, within noise), so the gains are interactions: better-optimised training is what lets the model use the $8\times$ larger effective data set. The FNO retains its advantage everywhere ($1.8\times$ better in the full recipe, vs. $1.5\times$ with the Stage 2 recipe). All later studies use the full recipe.

5 Study 2: Optimising the FNO Architecture

With the full recipe and 20 000 steps we varied one factor at a time around $(m, w, L) = (12, 64, 4)$, then trained joint candidates for 60 000 steps (Figure 2, Table 2).

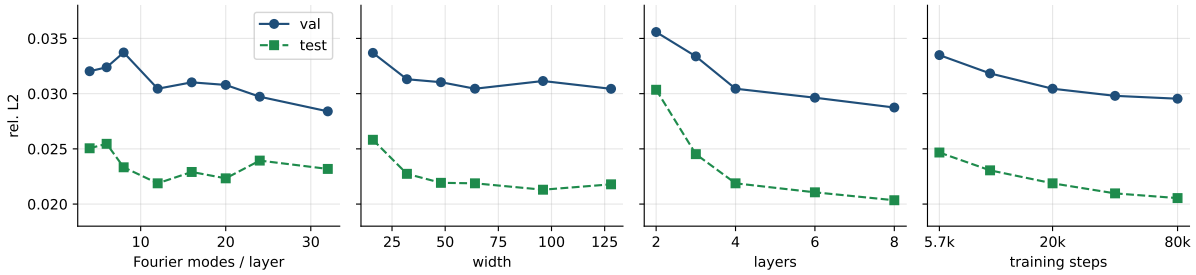


Figure 2: One-factor sweeps (single seed, full recipe, 20 000 steps except the step sweep). Blue: validation, green: test. Seed-to-seed sd is ≈ 0.0004.

Table 2: Selection by validation error. Left: joint candidates, 60 000 steps, one seed. Right: MLP capacity, 20 000 steps.

FNO (m, w, L)	params	val	test	
(12, 64, 4)	4.7 M	0.0298	0.0212	
(16, 64, 6)	12.6 M	0.0286	0.0210	
(16, 96, 6)	28.4 M	0.0281	0.0203	
(16, 96, 8)	37.8 M	0.0284	0.0198	
(12, 64, 8)	**9.5 M**	**0.0273**	0.0200	**selected**

MLP hidden	params	val	test
2×1024	17.8 M	0.0513	0.0463
3×1024	18.9 M	0.0469	0.0412
3×2048	42.0 M	0.0477	0.0399
5×2048	50.4 M	0.0509	0.0462
3×4096	**100.7 M**	**0.0461**	0.0404

Findings. (i) The FNO is remarkably insensitive to width (≥ 32) and to the number of retained modes (≥ 8; the whole $m \in [4, 32]$ range spans only 0.022–0.025 test error, while parameters grow from 0.5 M to 33.6 M). (ii) Depth and training length help slowly: $L = 2 \to 8$ gives $0.030 \to 0.020$; 5 700→80 000 steps gives $0.0247 \to 0.0205$. (iii) Learning rate ($5{\times}10^{-4}$–$2{\times}10^{-3}$), weight decay (0–10^{-2}) and batch size (8–32) change the test error by < 0.001. (iv) The best validation error selects a deeper, *not wider* model, $(12, 64, 8)$; larger models are not better on validation. The differences among the top candidates are of the order of the seed noise, so we chose the smallest model within it. The MLP was tuned on the same footing; its validation-best is 3×4096 (100 M parameters), and larger MLPs do not fix its gap. *Final models* were retrained with 5 seeds for 60 000 steps:

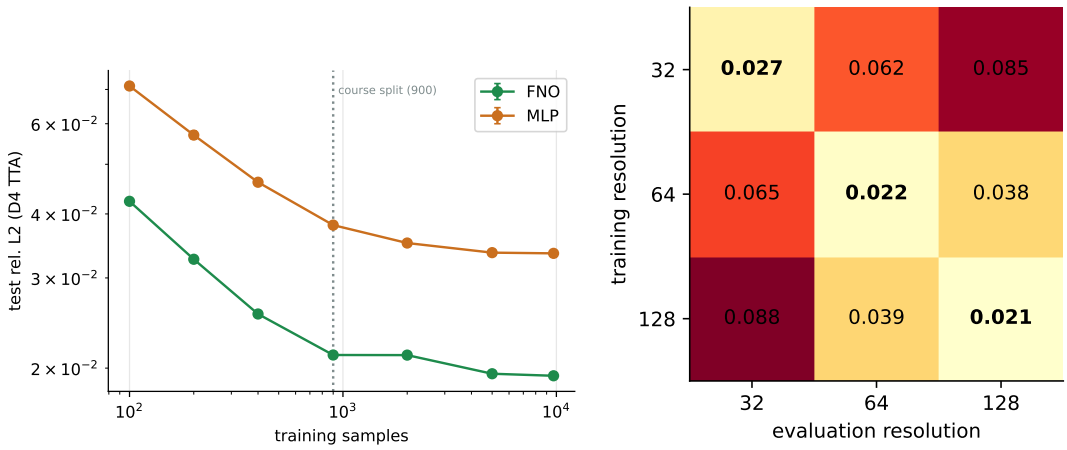
Table 3: Final models (60 000 steps, full recipe, 5 seeds, test set of 200 fields).

model	params	mean rel. L_2	median	with 2-elem. TTA
Stage 2 FNO (reported)	4.7 M	0.0521	0.0398	–
Stage 2 MLP (reported)	42.0 M	0.0820	0.0695	–
Final FNO	9.5 M	$\mathbf{0.0202 \pm 0.0004}$	0.0070	0.0200 ± 0.0003
Final MLP	100.7 M	0.0415 ± 0.0009	0.0272	0.0386 ± 0.0009

Training the final FNO takes ≈ 32 min on one shared GPU. The FNO error is 2.6× lower than the Stage 2 FNO and 2.05× lower than the best MLP, with 10.6× fewer parameters. The median FNO error is only 0.7%: the mean is dominated by a small tail (§8).

6 Study 3: Data Scaling and Resolution

Data scaling. We trained FNO $(12, 64, 4)$ and the 3×2048 MLP with $n \in \{100, \ldots, 9700\}$ samples (900 course samples plus the unused real data; fixed 20 000 steps, 2 seeds each, TTA).



(a) Data scaling **(b)** Train vs. evaluation resolution

Figure 3: (a) FNO error falls from 0.042 ($n = 100$) to 0.0195 ($n = 5000$) and then saturates; at 200 samples the FNO is at 0.033 vs. 0.057 for the MLP, which plateaus at 0.034. (b) Test error of FNOs trained at one resolution and evaluated at another (2 seeds).

The FNO is much more sample-efficient: with 400 samples it is already better than the MLP with 9 700, and at $n = 900$ the ratio is 1.8×. The FNO plateau at ≈ 0.019–0.020 for $n \geq 5000$ cannot be broken by more data at this resolution and model size, which points to a resolution/representation floor rather than a data floor (see §8).

Resolution. FNOs trained at 32, 64 and 128 and evaluated at all three resolutions (Fig. 3b). In-resolution errors are 0.027, 0.022, 0.021: refining from 64 to 128 gains only $\approx 4\%$ while costing 4× more compute. Zero-shot transfer works but is far from free: a model trained at 64 attains 0.038 at 128 (the final 60 000-step model: 0.037) and 0.065 at 32 (final: 0.063), i.e. 1.7–3× worse than a native-resolution model; coarse-trained models transfer worst (0.085 at 128). Zero-shot super-resolution is therefore useful for cheap grid refinement, but "resolution-independent" should be read as *applicable* at any resolution, not *accurate* at any resolution: the input statistics and discretisation error of the target change with the grid.

7 Study 4: Comparison with a Finite-Volume Solver and Inference Cost

A correction. Stage 2 reported FNO speed-ups of 1443× and 7740× relative to a 1030 ms FDM solve. That reference used a solver whose boundary rows were set in a Python loop over sparse-matrix rows, which is orders of magnitude slower than a competent implementation; *those speed-up figures are withdrawn.* We wrote a vectorised cell-centred finite-volume solver (`src/solvers.py`: COO assembly, sparse LU, and algebraic-multigrid-preconditioned CG via `pyamg`); the two solvers agree to 10^{-9}.

Which discretisation reproduces the ground truth? Ideal cell-centred FV (Dirichlet boundary half a cell outside the array) differs from PDEBench's arrays by 5–10%, because the reference data behave as if the Dirichlet boundary were about one cell outside the grid. We therefore calibrated the face averaging and boundary distance b per resolution on 40 validation fields (Table 4), i.e. we give the classical solver the same advantage of tuning to the data that the FNO gets by training on it. Coarse grids use stride sub-sampling of both κ and u, exactly as the FNO sees them.

Table 4: FV solver vs. ground truth (test set) and cost (median of 40 solves, 1 CPU thread).

grid	calibration	err. strict FV	err. calibrated	direct LU [ms]	AMG-CG [ms]
32×32	harmonic, b=0.7	0.098	0.094	1.8	5.5
64×64	harmonic, b=0.8	0.059	0.044	7.1	11.6
128×128	arithmetic, b=1.05	0.051	0.024	37.4	30.7

Latency. We timed the final models with CUDA events on an idle GPU and with wall-clock on a 16-core CPU (CPU numbers are indicative only).

Table 5: Inference time per solution at 64×64 (median over repetitions). b = batch size, smp = sample; GPU batch-1 latency is kernel-launch bound.

	test err.	GPU, b=1 [ms]	GPU, b=128 [ms/smp]	CPU8, b=32 [ms/smp]
Final FNO (9.5 M)	0.0202	1.44	0.20	16.5
FNO 4-layer (4.7 M)	0.0212	0.79	0.106	–
Final MLP (100.7 M)	0.0415	0.27	0.0058	0.57
FV solver, N=64	0.044	7.1 ms (1 CPU thread)		
FV solver, N=128	0.024	37.4 ms (1 CPU thread)		

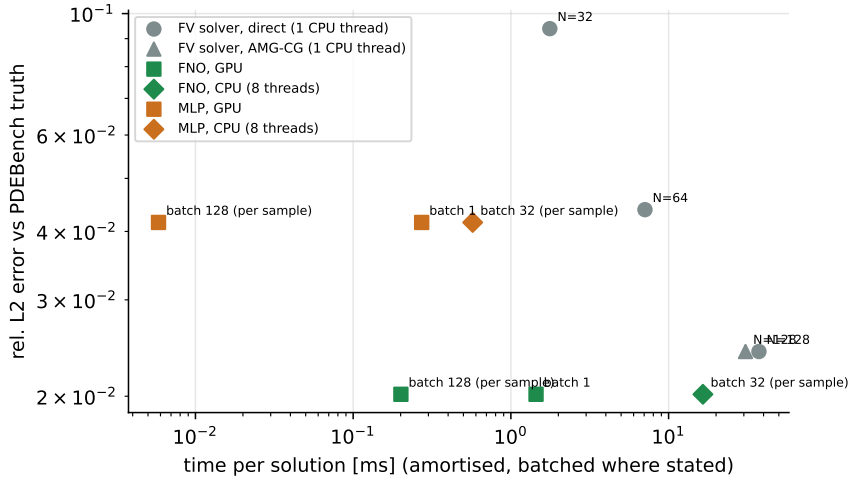


Figure 4: Error–time trade-off. Solver: 1 CPU thread. FNO/MLP: GPU or 8-thread CPU as marked.

Reading the comparison. (i) At the same 64×64 resolution the FNO agrees with the ground truth 2.2× better than the calibrated solver (0.020 vs. 0.044) and is 5× (batch 1) to 35× (batched) faster on a GPU; it is even more accurate than the 128×128 solver (0.024) while 26–187× faster. (ii) The FNO's advantage in *agreement with the data* is partly because it learns the data-generator's own discretisation quirks (the effective boundary offset) that a standard FV scheme does not share; it is a statement about this dataset, not a claim of higher fidelity to the continuous PDE. (iii) On one CPU core the FNO is *not* faster: 37 ms at batch 1, versus 7 ms for the 64^2 solver. The surrogate pays off through GPU batching and amortisation: training costs ≈ 32 min ($\approx 1.9 \times 10^3$ s), so it breaks even with the 128^2 solver after $\approx 5 \times 10^4$ solves on one core. (iv) The MLP is the fastest model (0.006 ms/sample) but 2× less accurate than the FNO. (v) TF32 does not accelerate the FNO (1.47 vs. 1.44 ms), and `bfloat16` autocast is unsupported by PyTorch for the complex FFT operations, so mixed precision is not an easy lever here.

8 Physical Interpretation

All diagnostics use the seed-0 final FNO on the 200 test fields (`src/stage3_diagnostics.py`).

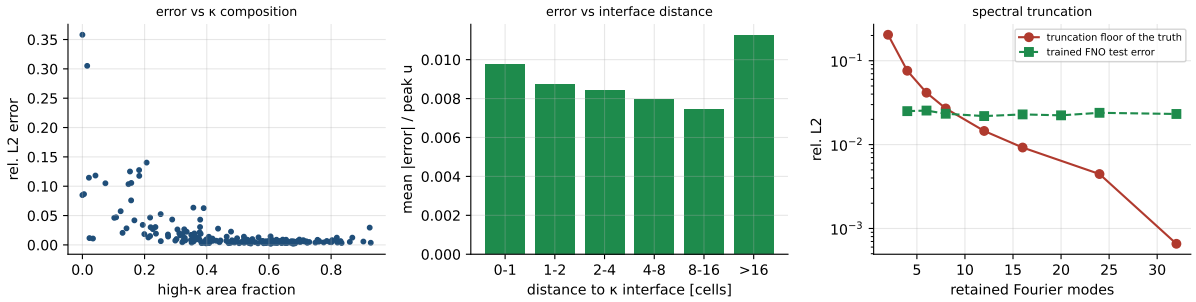


Figure 5: Left: error vs. high-permeability area fraction. Middle: mean absolute error (relative to the field's peak pressure) vs. distance to the nearest κ interface. Right: relative error of the truth when truncated to m Fourier modes (red) vs. the test error of FNOs actually trained with m modes (green).

Where the error is. The median error is 0.7% but the mean is 2.0%: the two exactly uniform-κ test fields have errors 0.36 and 0.085, and excluding them the mean falls to 1.8%; the 90th percentile is 4.6%. Errors correlate with the peak pressure ($r = 0.61$) and negatively with the high-κ area fraction ($r = -0.55$): low-permeability-dominated fields have the largest pressure amplitudes because $u \propto 1/\kappa$, and a network that must infer the overall amplitude from a near-constant input has the least information about it (Figure 6, bottom row: correct shape, wrong amplitude). Contrary to the common expectation that neural surrogates fail at material interfaces, the error is only mildly larger at interfaces (0.0098 in the 0–1 cell band) than 8–16 cells away (0.0075), and largest in the interior of large regions (0.0112 beyond 16 cells), i.e. where the pressure peaks.

Does the prediction obey the physics? The relative residual of the finite-volume operator applied to the FNO prediction is 4.25, versus 4.01 for the ground truth itself: the residual is large in both because the truth is only a second-order accurate solution on the sub-sampled grid, but the FNO prediction is within 6% of the truth's own residual, so it satisfies the discrete equation as well as the reference does at this resolution. The mean of the pressure field (an integral flux-balance quantity) is reproduced to 1.7%; only 1.5×10^{-5} of the predicted pixels are negative (the maximum principle requires $u \geq 0$); and the mean value on the outermost pixel ring relative to the interior is 0.1157 (truth: 0.1159), so the Dirichlet behaviour is learned rather than imposed.

Spectral view. The truth truncated to $m = 4/8/12/16$ modes has relative error 7.6/2.7/1.5/0.9%, so at least ≈ 12 modes are needed to represent the solution linearly in Fourier space. Yet an FNO trained with only 4 modes reaches 2.5%: the spectral weights are not a hard band limit, because the 1×1 bypass, the pointwise nonlinearities and depth regenerate high-frequency content. Beyond ≈ 8 modes the accuracy is insensitive to m (Fig. 5, right), and so the observed ≈ 0.02 plateau is set by the amplitude/generalisation problem above and the input discretisation, not by the number of modes.

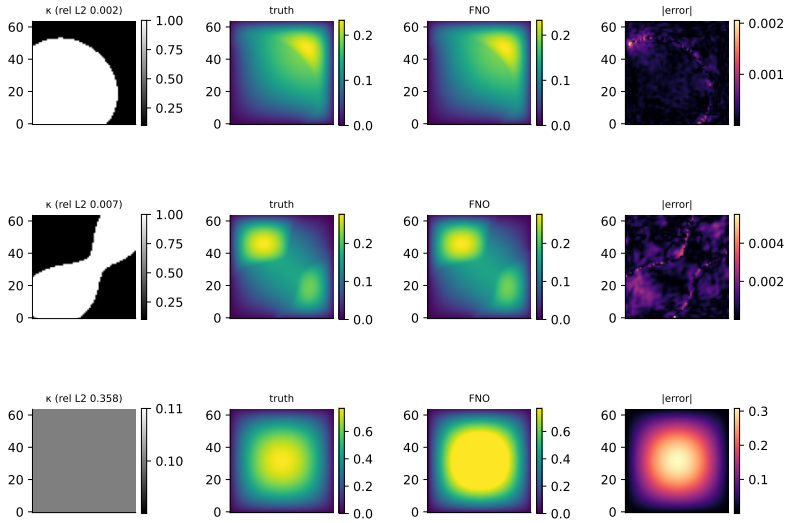


Figure 6: Best, median and worst test sample of the final FNO: κ (relative error in title), truth, prediction, absolute error. The worst case has uniform $\kappa = 0.1$.

9 Study 5: Noisy Labels

The data are exact solver output of a simple linear PDE, so fitting them is easy; measured pressures would not be exact. We therefore add fixed Gaussian noise to the *training and validation* pressure maps, with noise RMS equal to a fraction $s \in \{1, 5, 10\}$ % of each map's own RMS, and keep the *test* maps clean, so the reported error is the error against the true field. Checkpoints are selected on the noisy validation maps (all that would be available in practice). Everything else is the full recipe of §4 (5 700 steps, three seeds, both models; `src/stage3_robust.py`, run on Kaggle T4 GPUs).

Table 6: Test relative L_2 error on clean maps after training on noisy labels (mean ± sd, 3 seeds).

label noise	FNO	MLP	MLP / FNO
none	0.0249 ± 0.0002	0.0454 ± 0.0002	1.83×
1 %	0.0262 ± 0.0002	0.0453 ± 0.0004	1.73×
5 %	0.0298 ± 0.0035	0.0466 ± 0.0007	1.56×
10 %	0.0362 ± 0.0011	0.0509 ± 0.0008	1.41×



Figure 7: Test error on clean maps vs. label noise; the dashed line is the error of the noisy labels themselves.

The FNO error rises by +5 %, +20 % and +45 % at 1, 5 and 10 % noise, the MLP error by 0 %, +3 % and +12 %. The FNO stays ahead of the MLP at every noise level, but the gap narrows from 1.8× to 1.4×. Both models average the noise out: with labels that are 10 % from the truth, the FNO is 3.6 % and the MLP 5.1 % from the clean field. Caveats: the noise is i.i.d. Gaussian and relative to each map, the inputs κ are kept clean, and the 5 % FNO result has one seed noticeably worse than the other two (hence the larger sd). A plausible reason that the MLP is less sensitive is that its error is limited by architecture rather than by the labels (its clean error is already high and flat across training-set size, §6); we did not test this.

10 Study 6: Self-Adjusting Sample Weights

The error is concentrated in a few low-permeability, almost uniform fields (§8). Following the instructor's suggestion we tried self-adjusting weights: every training map i keeps a running (EMA, momentum 0.9) estimate e_i of its own loss, updated whenever it is seen, and its loss weight is $w_i = \mathrm{clip}\big((e_i/\bar{e})^a, 0.2, 5\big)$, renormalised to mean 1, so maps that are currently fitted badly count more ($a = 0$ is the ordinary loss; we tried $a = 1, 2$). Setup as in Study 5 without noise (full recipe, 5 700 steps, three seeds). "Hard" maps are those with less than 5 % high-permeability area: 8 of the 200 test maps and 27 of the 900 training maps.

Table 7: Self-adjusting weights, FNO test error (mean ± sd, 3 seeds).

loss	all 200 maps	192 other maps	8 hard maps
plain	0.0249 ± 0.0002	0.0189 ± 0.0004	0.170 ± 0.008
adaptive, $a = 1$	0.0265 ± 0.0006	0.0204 ± 0.0005	0.172 ± 0.005
adaptive, $a = 2$	0.0277 ± 0.0006	0.0217 ± 0.0002	0.170 ± 0.010

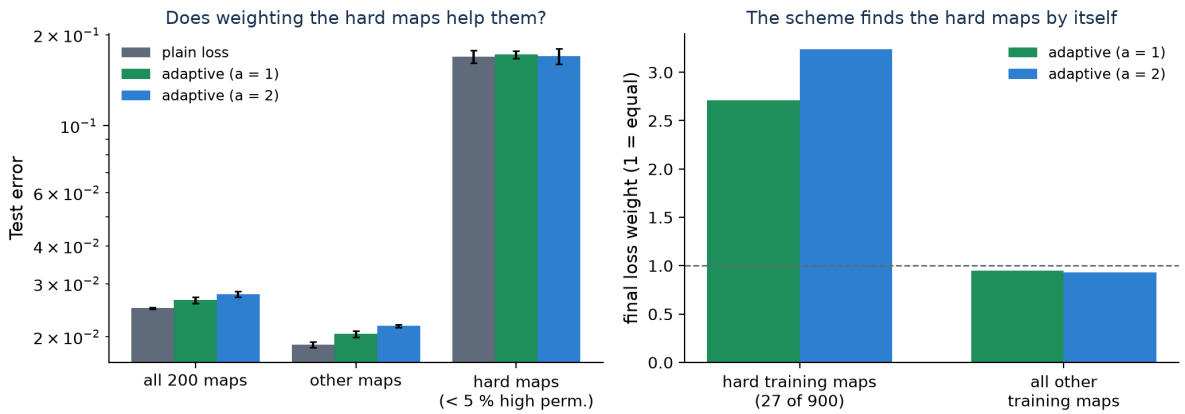


Figure 8: Left: test error overall, on the other maps and on the 8 hard maps. Right: final loss weight of the 27 hard and the other training maps.

The mechanism works: the 27 hard training maps end with a mean weight of 2.7 ($a = 1$) and 3.2 ($a = 2$) against about 0.95 for all others, and the weight correlates with the high-permeability share at $r \approx -0.6$, so the scheme finds the hard maps without being told. But the result is negative: the error on the hard test maps does not improve (0.170 → 0.172 ($a = 1$), 0.170 ($a = 2$), well within the seed spread), while the error on the other maps rises by +8 % and +15 % and the overall error by +6 % and +11 %. Our interpretation, not tested here, is that the limit is information rather than weighting: only 27 of 900 training maps are of this kind, and giving them more weight cannot supply the missing examples; more such maps or a scale-normalised target (Limitations) are the natural next steps. The evidence is limited to 8 hard test maps, one weighting rule and two exponents.

11 Limitations and Honest Notes

- **One PDE, one dataset, one forcing** ($f = 1$, binary κ); conclusions about scaling and plateau may not transfer to other operators or resolutions. Model selection used one seed per candidate; differences between top architectures are within seed noise.
- **Stage 2 numbers.** The Stage 2 recipe (MSE loss, per-step cosine restarts) was sub-optimal; the Stage 2 comparison was fair (same recipe for both models) but its absolute errors were not the best achievable. Stage 2's speed-up figures are withdrawn (§7). The earlier "78.8 M-parameter scaled FNO" result (0.046) is superseded: a 9.5 M model is 2.3× better.
- **Solver fairness.** The solver was calibrated on validation data to reproduce PDEBench's arrays; the comparison measures agreement with this dataset. Solver timings use a single CPU thread without a compiled multigrid; a GPU or highly tuned solver would narrow the gap. The 2-element TTA costs 2× the latency and is not included in Table 5; it changes the final FNO's error by < 0.0003.
- **Uniform κ.** The two degenerate uniform-κ fields dominate the tail. Because $u \propto 1/\kappa$ for a uniform field, a scale-normalised formulation (predict κu) might remove this failure; we did not test it. Self-adjusting sample weights (§10) did not reduce the error on these fields.
- **Robustness studies** (§9–§10) use the 5 700-step recipe, three seeds, one noise type (relative i.i.d. Gaussian on the pressure labels only) and, for the weighting, one scheme with two exponents and 8 hard test maps; they ran on Kaggle T4 GPUs, whereas the main study ran on a different GPU (the clean baseline reproduces: 0.0243 vs. 0.0247).
- **Timing** was measured on a shared machine; CPU numbers are indicative. Training used

concurrent jobs on shared GPUs, so reported steps/s in the run logs are not benchmarks.

12 Conclusion

On real PDEBench Darcy data the FNO outperforms an equally tuned MLP by 2× in accuracy with 10× fewer parameters, and by 2.2× in agreement with the data over a calibrated finite-volume solver at the same resolution, while being 5–35× faster on a GPU (but not on a CPU core). The dominant lever was not architecture but training: the loss matching the metric, a single learning-rate decay and symmetry augmentation halved the error; width, modes, learning rate and weight decay hardly matter beyond modest values; and the error plateaus near 2% once $\gtrsim$ 5,000 samples are available. Residual errors are concentrated in low-contrast, high-amplitude fields, not at material interfaces, and predictions respect the boundary condition, positivity and the discrete PDE as well as the reference data do.

Two checks added after review: with up to 10 % noise on the training labels the FNO still beats the MLP at every level (clean-test error 0.025 → 0.036 vs. 0.045 → 0.051), and self-adjusting sample weights found the hardest fields but did not reduce their error.

Reproducibility

- Scripts in `src/`: `stage3_train.py` (training, evaluation), `run_jobs.py` (job queue), `solvers.py` and `solver_vs_fno.py` (FV solver, calibration, timing).
- `stage3_latency.py` (latency), `stage3_diagnostics.py` (physics), `stage3_aggregate.py`, `stage3_figures.py`.
- Robustness studies: `stage3_robust.py`, `stage3_robust_aggregate.py`, job list `scripts/stage3_jobs/jobs_extra.txt`, results in `results/stage3_extra/`.
- Job lists in `scripts/stage3_jobs/`; all per-run metrics in `results/stage3/runs/*/metrics.json`; commands and the table/figure→script map in `stage3/README.md`.

References

[1] Z. Li et al., *Fourier Neural Operator for Parametric Partial Differential Equations*, ICLR 2021.

[2] M. Takamoto et al., *PDEBench: An Extensive Benchmark for Scientific Machine Learning*, NeurIPS Datasets and Benchmarks, 2022.

[3] N. Bell, L. Olson, J. Schroder, *PyAMG: Algebraic Multigrid Solvers in Python*, JOSS 2022.